

L4c: Shortest-Path Algorithms

CHEME 5800 Cornell University

Objectives for Today

Today we formulate the single-source shortest-path problem and compare two global schedules for the same local relaxation update.

Today's concepts

- **Shortest paths and relaxation**: calculate path cost and update distance and predecessor maps.
- **Dijkstra's algorithm**: explain greedy finalization under nonnegative edge weights.
- **Bellman-Ford**: explain repeated relaxation, negative edges, and reachable negative-cycle detection.

Lecture: [Shortest-Path Algorithms](#)

Lecture and Lab

Lecture: Shortest-Path Algorithms

Formulate the problem, trace Dijkstra and Bellman-Ford, and distinguish a permissible negative edge from a reachable negative-weight cycle.

Lab L4d: Production Planning with Shortest Paths

Predict a least-cost production route, compute it with Dijkstra, verify it independently with Bellman-Ford, and locate a cost threshold where the optimum changes.

Weighted Graphs Turn Routes into Optimization

An edge weight can represent travel time, communication delay, monetary cost, or the number of unit operations in a chemical process.

The common question is:

Among all feasible routes from a source to a target,
which route has the **smallest total edge weight**?

Enumerating every route is usually impractical because the number of routes may grow exponentially with graph size.

A Feasible Route Follows Directed Edges

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a finite directed graph with vertex set $\mathcal{V}$ and edge set $\mathcal{E}$. The function $w : \mathcal{E} \rightarrow \mathbb{R}$ assigns each edge a real-valued cost.

We write $|\mathcal{V}|$ for the number of vertices and $|\mathcal{E}|$ for the number of edges. A route P from source s to target t with k edges has the ordered vertices:

$$P = \langle v_0, v_1, \dots, v_k \rangle, \quad v_0 = s, \quad v_k = t,$$

with $(v_i, v_{i+1}) \in \mathcal{E}$ for $i = 0, \dots, k-1$.

Its total weight is the sum of its edge weights:

$$\underbrace{w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})}_{\text{route cost}}.$$

Distance Is Finite Only for a Reachable Target

Let $\mathcal{P}_{s,t}$ be the set of feasible routes from s to t . The $\min$ operator selects the smallest route cost. If no negative-weight cycle lies on an s -to- t route, define:

$$d(s, t) = \begin{cases} \min_{P \in \mathcal{P}_{s,t}} w(P), & \mathcal{P}_{s,t} \neq \emptyset, \\ +\infty, & \mathcal{P}_{s,t} = \emptyset. \end{cases}$$

An unreachable target has $d(s, t) = +\infty$. For a finite minimum, $\arg \min$ gives the set of routes attaining it. Select one such route P^* :

$$P^* \in \arg \min_{P \in \mathcal{P}_{s,t}} w(P), \quad w(P^*) = d(s, t).$$

A Single-Source Solution Returns Two Maps

Fix source s and compute, for every vertex $v \in \mathcal{V}$:

$$\underbrace{\text{dist}[v] = d(s, v)}_{\text{minimum route cost}}, \quad \underbrace{\text{prev}[v]}_{\text{one predecessor on a selected route}} .$$

- **nothing** means no predecessor is recorded; the source has $\text{prev}[s] = \text{nothing}$.
- Follow predecessors backward from a target, then reverse the sequence, to reconstruct one minimizing route.
- If several routes tie, the predecessor map may encode any one of them.
- An unreachable vertex keeps distance $+\infty$ and no predecessor.

Shortest Paths Have Optimal Substructure

Suppose a shortest path from source s to target t passes through vertex u .

Its prefix from s to u must itself be a shortest path.

If a cheaper prefix existed, replacing the old prefix would produce a cheaper route to t , contradicting optimality.

A globally optimal route is assembled from optimal prefixes.

Algorithmic consequence: store the best discovered prefix to each vertex instead of enumerating complete routes.

Relaxation Starts with One Known Route

At initialization, the zero-edge route from s to itself is the only known route:

$$\text{dist}[v] = \begin{cases} 0, & v = s, \\ +\infty, & v \neq s, \end{cases} \quad \text{prev}[v] = \text{nothing}.$$

A finite estimate $\text{dist}[v]$ is always the weight of an actual source-to- v route discovered so far.

These estimates begin conservatively and can only decrease as better routes are found.

Relax One Extended Route

For edge $(u, v) \in \mathcal{E}$, extend a known finite route to u . We use $\text{alt}(u, v)$, short for **alternative route cost**, for the cost of reaching v this way:

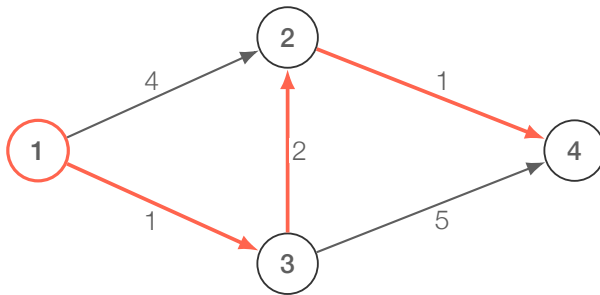
$$\underbrace{\text{alt}(u, v) = \text{dist}[u] + w(u, v)}_{\text{candidate route cost to } v}.$$

Relaxation updates both maps only when the candidate is strictly cheaper:

$$(\text{dist}[v], \text{prev}[v]) \leftarrow \begin{cases} (\text{alt}(u, v), u), & \text{alt}(u, v) < \text{dist}[v], \\ (\text{dist}[v], \text{prev}[v]), & \text{alt}(u, v) \geq \text{dist}[v]. \end{cases}$$

In the algorithm, alt stores this cost for the current edge. Tied costs leave the predecessor unchanged.

Example: A Four-Vertex Graph



Edge labels give costs. Red edges mark the minimum-cost route from source 1 to target 4. We first examine how relaxation improves the estimate for vertex 2.

Example: Relaxing One Edge

From source 1, the direct edges give $\text{dist}[2] = 4$ and $\text{dist}[3] = 1$. Both vertices initially have predecessor 1.

Can we reach vertex 2 more cheaply through vertex 3? For edge $(3, 2)$, the alternative route cost is:

$$\text{alt}(3, 2) = \text{dist}[3] + w(3, 2) = 1 + 2 = \textcolor{red}{3}.$$

Since $3 < 4$, we update both maps:

$$\text{dist}[2] \leftarrow 3, \quad \text{prev}[2] \leftarrow 3.$$

Following predecessors from vertex 2 leads to vertex 3 and then vertex 1. Reversing this sequence gives the cheaper route $1 \rightarrow 3 \rightarrow 2$. We still need to establish when a distance estimate is final.

Two Useful Properties of Relaxation

For a vertex v with finite shortest-path distance, a finite estimate $\text{dist}[v]$ is the cost of a discovered route. Hence:

$$\underbrace{d(s, v) \leq \text{dist}[v]}_{\text{upper-bound invariant}}.$$

For an edge (u, v) with distinct endpoints, so $u \neq v$, immediately after relaxation:

$$\underbrace{\text{dist}[v] \leq \text{dist}[u] + w(u, v)}_{\text{local consistency for } (u, v)}.$$

A later decrease in $\text{dist}[u]$ can make this edge worth relaxing again. The algorithm decides when to revisit edges and when distances are final.

Dijkstra and Bellman-Ford use the same relaxation rule.
They differ in the order, repetition, and assumptions surrounding that update.

Dijkstra Requires Nonnegative Edge Weights

Dijkstra repeatedly selects the unsettled vertex with the smallest tentative distance. Its greedy step requires:

$$w(u, v) \geq 0 \quad \text{for every } (u, v) \in \mathcal{E}.$$

Initialize:

- distance and predecessor maps;
- an empty settled set $\mathcal{S}$;
- a min-priority queue $\mathcal{Q}$ containing s with priority zero.

Every other distance begins at $+\infty$. The public implementation rejects a graph containing any negative edge before running the greedy algorithm.

Dijkstra Finalizes One Vertex at a Time

While the min-priority queue $\mathcal{Q}$ is not empty:

1. Remove a vertex u with minimum priority.
2. If $u \in \mathcal{S}$, skip it; otherwise add u to $\mathcal{S}$. Its distance is now final.
3. For every outgoing edge (u, v) , compute $\text{alt} = \text{dist}[u] + w(u, v)$.
4. If $\text{alt} < \text{dist}[v]$, update the distance, set $\text{prev}[v] = u$, and assign priority alt to v .

A weighted adjacency list supplies the outgoing target and weight pairs needed in step 3.

Dijkstra Trace on the Worked Graph

Each row records the distance estimates after the indicated step.

Step	Action	dist[1]	dist[2]	dist[3]	dist[4]
0	initialize at source 1	0	∞	∞	∞
1	settle 1; relax (1, 2) and (1, 3)	0	4	1	∞
2	settle 3; improve 2 and discover 4	0	3	1	6
3	settle 2; improve 4	0	3	1	4
4	settle 4	0	3	1	4

The predecessor updates are $\text{prev}[3] = 1$, $\text{prev}[2] = 3$, and $\text{prev}[4] = 2$. Following them backward reconstructs $4 \leftarrow 2 \leftarrow 3 \leftarrow 1$.

Why Is a Settled Distance Final?

With nonnegative weights, the source is correctly settled at distance zero. Assume all previously settled distances are correct. Let u be selected next and suppose a cheaper route to u exists.

On that route, let y be the first unsettled vertex and x its settled predecessor. Since $\text{dist}[x] = d(s, x)$, relaxing (x, y) gave y an estimate no greater than the route prefix to y .

The remaining edges have nonnegative weights, so the prefix costs no more than the full route. Thus y would have a smaller estimate than u , contradicting the selection rule:

$$u \in \arg \min_{v \notin \mathcal{S}} \text{dist}[v] \implies \text{dist}[u] = d(s, u).$$

A Negative Edge Can Invalidate Finalization

Consider a different graph with source vertex 1 and three directed edges:

$$w(1, 2) = 2, \quad w(1, 3) = 5, \quad w(3, 2) = -4.$$

If we apply Dijkstra's selection rule despite the negative edge, vertex 2 is selected with distance 2 before vertex 3.

Yet the route through vertex 3 has lower cost:

$$1 \rightarrow 3 \rightarrow 2, \quad 5 - 4 = 1.$$

Thus, a selected distance need not be final. Our implementation rejects negative-edge inputs before applying the selection rule.

Dijkstra Uses a Queue to Scan Edges

With a weighted adjacency list and a binary-heap min-priority queue, Dijkstra runs in:

$$\underbrace{\mathcal{O}((|\mathcal{V}| + |\mathcal{E}|) \log |\mathcal{V}|)}_{\text{Dijkstra running time}}.$$

- Each reached vertex enters the queue as tentative distances improve.
- Each outgoing edge is examined when its source is settled.
- The settled set prevents a vertex from being finalized twice.

Scope: the bound matches the course implementation using adjacency lists and a binary-heap priority queue.

Bellman-Ford Repeatedly Relaxes Every Edge

Bellman-Ford permits negative edges by giving later improvements time to propagate across earlier vertices.

Initialize the same distance and predecessor maps. Then repeat at most $|\mathcal{V}| - 1$ times:

1. Visit every edge $(u, v) \in \mathcal{E}$.
2. If $\text{dist}[u]$ is finite and the candidate is cheaper, relax (u, v) .
3. If a complete pass changes nothing, stop early because the estimates converged.

Updates are available immediately to later edges in the same pass. The edge order can therefore change how quickly the estimates converge.

Bellman-Ford: Following One Pass

Starting at source 1, visit the edges in the order below. Each row shows the estimates after relaxing that edge; $\text{dist}[1] = 0$ throughout.

Edge relaxed	$\text{dist}[2]$	$\text{dist}[3]$	$\text{dist}[4]$
Initial state	∞	∞	∞
(1, 2)	4	∞	∞
(1, 3)	4	1	∞
(3, 2)	3	1	∞
(2, 4)	3	1	4
(3, 4)	3	1	4

Relaxing (3, 2) lowers the estimate for vertex 2 from 4 to 3. The next edge, (2, 4), uses that improvement immediately to give $3 + 1 = 4$.

Bellman-Ford: Comparing Edge Orders

Now reverse the edge order used in the preceding trace:

$(3, 4), (2, 4), (3, 2), (1, 3), (1, 2).$

Each triple gives $(\text{dist}[2], \text{dist}[3], \text{dist}[4])$ after a complete pass.

Passes completed	Listed edge order	Reversed edge order
0	(∞, ∞, ∞)	(∞, ∞, ∞)
1	$(3, 1, 4)$	$(4, 1, \infty)$
2	$(3, 1, 4)$; no change, stop	$(3, 1, 5)$
3	Already stopped	$(3, 1, 4)$

In the reversed order, $(2, 4)$ is visited before $(3, 2)$. An improvement at vertex 2 waits until the next pass to reach vertex 4.

Pass k Accounts for Paths with at Most k Edges

After pass k , Bellman-Ford has accounted for every source-to-vertex path containing at most k edges.

If no reachable negative-weight cycle exists, a shortest path can be chosen to be simple. Repeating a vertex would add either a nonnegative cycle or a removable zero-cost cycle.

A simple path visits at most $|\mathcal{V}|$ vertices and therefore contains at most:

$$\underbrace{|\mathcal{V}| - 1}_{\text{maximum edges in a simple path}}$$

This is why $|\mathcal{V}| - 1$ relaxation passes are sufficient.

One Extra Pass Detects Negative Cycles

After the usual passes, scan every edge once more.

If an edge from a finite-distance vertex can still be relaxed, there is a reachable cycle C with total weight $w(C) < 0$. For a route P through C , repeating the cycle gives costs:

$$w(P), \quad w(P) + w(C), \quad w(P) + 2w(C), \quad \dots \longrightarrow -\infty \quad \text{when } w(C) < 0.$$

No finite shortest-path solution exists for vertices affected by that cycle.

A negative cycle disconnected from source s does not invalidate the single-source result because its edges begin at infinite-distance vertices.

Bellman-Ford Trades Time for Flexibility

In the worst case, Bellman-Ford scans every edge during each of $|\mathcal{V}| - 1$ passes and during the cycle check:

$$\underbrace{\mathcal{O}(|\mathcal{V}| |\mathcal{E}|)}_{\text{running time}}, \quad \underbrace{\mathcal{O}(|\mathcal{V}|)}_{\text{distance and predecessor storage}}.$$

Early exit can reduce the actual number of passes when a complete edge scan produces no update.

The benefit is correctness with negative edges and explicit detection of reachable negative-weight cycles.

Choose the Algorithm from the Weight Contract

Question	Dijkstra	Bellman-Ford
Edge weights	All nonnegative	Negative edges allowed
Global schedule	Settle minimum tentative vertex	Relax every edge repeatedly
Negative-cycle detection	Not supported	Detects a reachable negative-weight cycle
Worst-case time	$\mathcal{O}((\mathcal{V} + \mathcal{E}) \log \mathcal{V})$	$\mathcal{O}(\mathcal{V} \mathcal{E})$
Natural storage	Weighted adjacency list	Edge list

Both return distance and predecessor maps when their input contracts hold.

Different Schedules Produce the Same Optimum

On the nonnegative worked graph:

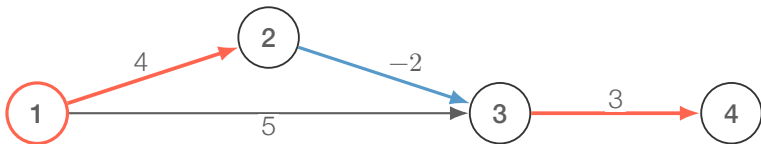
$$\text{dist}_{\text{Dijkstra}}[v] = \text{dist}_{\text{Bellman-Ford}}[v] \quad \text{for every vertex } v.$$

Both predecessor maps reconstruct the unique target route:

$$P^* = \langle 1, 3, 2, 4 \rangle, \quad w(P^*) = 1 + 2 + 1 = 4.$$

Agreement is expected because Dijkstra's nonnegative-weight precondition holds. Bellman-Ford provides an independent check using a different relaxation schedule.

A Negative Edge Can Still Have a Finite Optimum

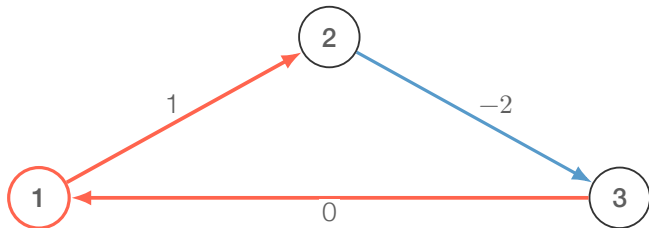


Bellman-Ford finds the finite route:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4, \quad 4 - 2 + 3 = 5.$$

A negative edge is permissible when no reachable negative cycle can be repeated.

Negative Cycles Remove Finite Optima



The cycle weight is:

$$w(C) = 1 - 2 + 0 = -1.$$

Each repetition lowers the route cost by one. Bellman-Ford's extra pass detects another possible relaxation and reports that no finite solution exists.

The Public Contracts Separate Three Cases

The notebook turns the comparison into executable checks:

- **Nonnegative graph:** both algorithms return $\langle 1, 3, 2, 4 \rangle$ with target cost 4 and identical distance maps.
- **Negative edge, no negative cycle:** Bellman-Ford returns $\langle 1, 2, 3, 4 \rangle$ with target cost 5.
- **Violated contract:** Dijkstra raises an error for the negative edge; Bellman-Ford raises an error for the reachable negative cycle.

Purpose: the tests verify numerical results and the public-interface assumptions, not only one displayed example.

Lab L4d Applies Both Algorithms

The lab maps alternative production routes to a weighted directed graph.

1. Predict the least-cost route by inspecting the process costs.
2. Compute the route with Dijkstra because all costs are nonnegative.
3. Verify the distance and route independently with Bellman-Ford.
4. Discount one production step and locate the break-even cost where the selected route changes.

Sensitivity analysis asks when the optimum changes, not only what the optimum is today.

Summary

We developed two algorithms for finding minimum-cost routes and examined how edge weights determine which algorithm we can use.

Key takeaways

- **Route costs and relaxation:** We summed edge weights to calculate route costs, improved distance estimates by relaxation, and used predecessors to reconstruct routes.
- **Dijkstra's algorithm:** We used a priority queue to select the smallest distance estimate and showed why nonnegative weights make it final.
- **Bellman-Ford's algorithm:** We used repeated relaxation to handle negative edges and an extra scan to detect reachable negative-weight cycles.

In [the L4d lab](#), we compare production routes, where fewer steps may not mean a lower total cost.

That's a wrap. Which property of the edge weights tells you which shortest-path algorithm is valid?